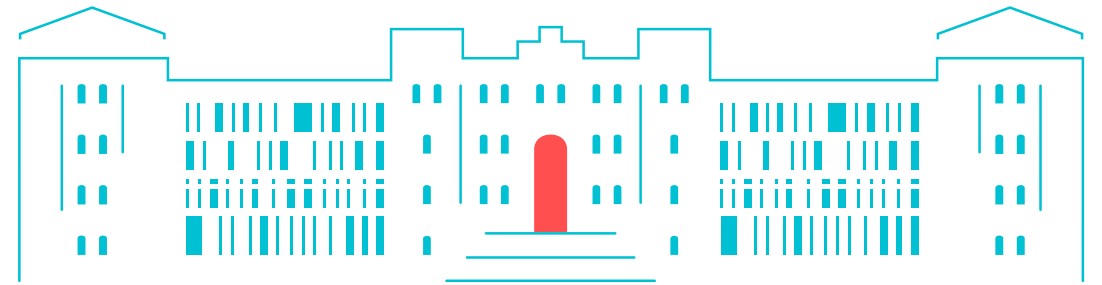
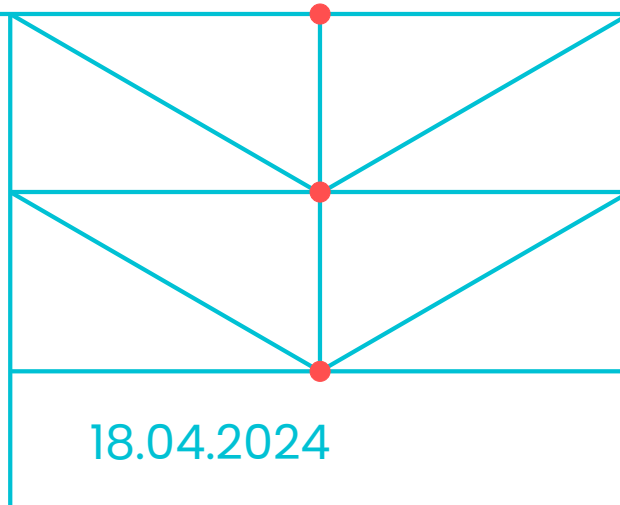


Parallelization of Solvers

Fynn Bense

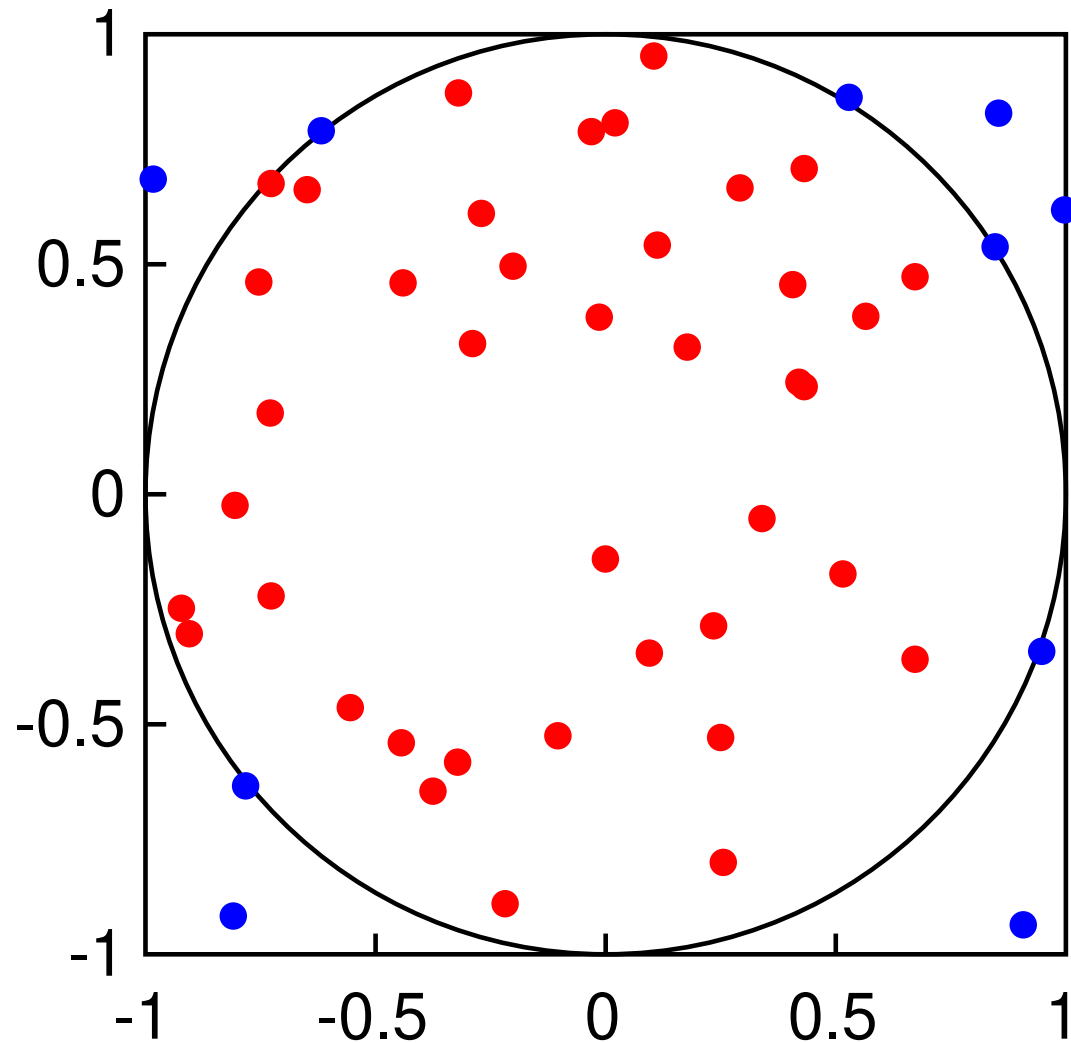
Chair Computational Mathematics
Institute of Mathematics (E-10)
Building E, Room 3.045

TUHH
Hamburg
University of
Technology



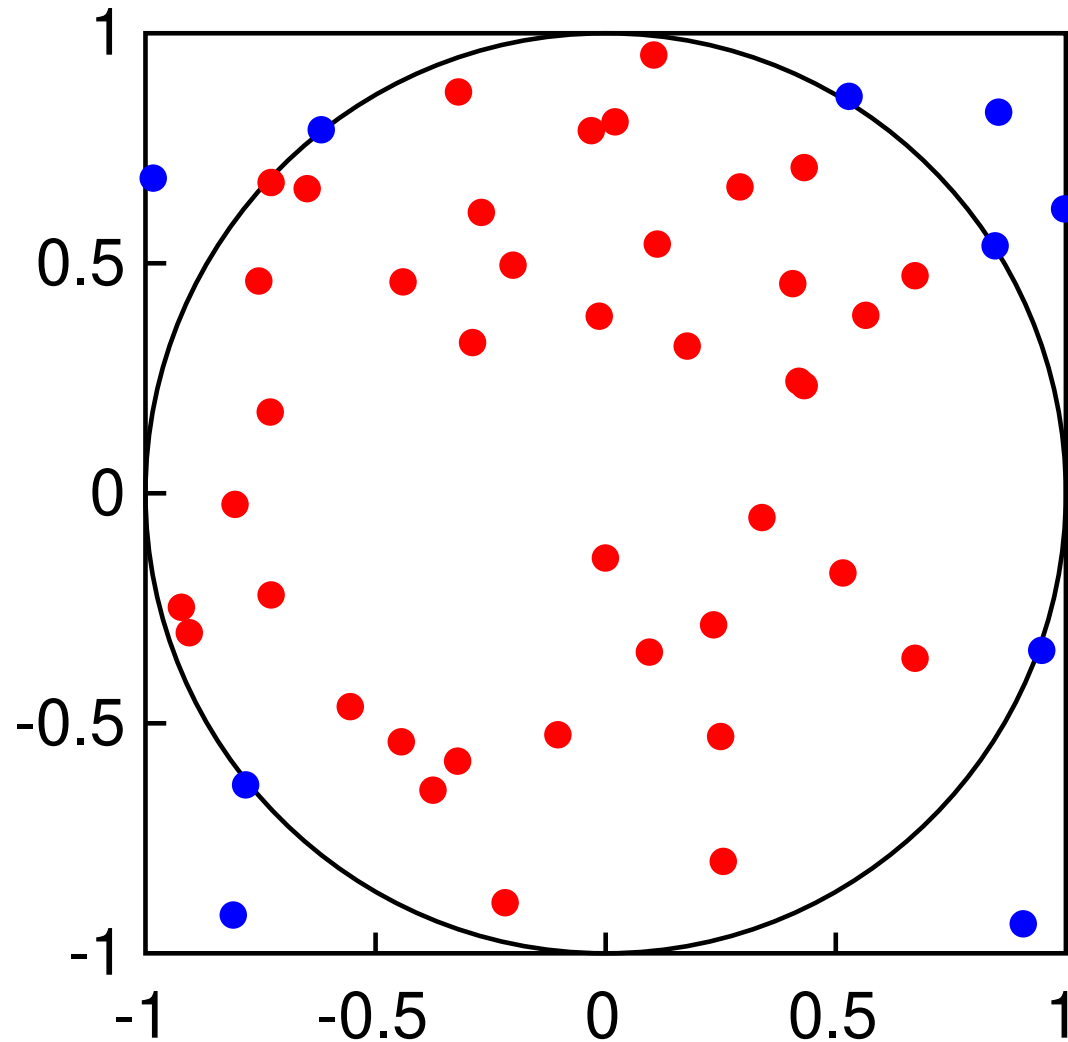
03 – Monte Carlo

- Monte Carlo approximation of π parallelized with OpenMP
- Thread safety
- What does NUMA mean?
- First touch policy



- Throw “darts” at the quarter circle and count number of hits vs. total number of attempts.
- Then,

$$V_{\text{ratio}} = \frac{\#hits}{\#attempts} = \frac{\pi r^2}{4}$$
$$\Rightarrow \pi \approx 4 \frac{\#hits}{\#attempts}$$



- Throw “darts” at the quarter circle and count number of hits vs. total number of attempts.
- Then,

$$V_{\text{ratio}} = \frac{\#hits}{\#attempts} = \frac{\pi r^2}{4}$$
$$\Rightarrow \pi \approx 4 \frac{\#hits}{\#attempts}$$

- The more attempts, the better the approximation should be
- Attempts can be run independently: this is called “embarrassingly parallel”

Monte Carlo estimation of π : a sequential code

```
1  for (i=0; i < attempts; i++)
2  {
3
4
5      /*
6      *   Throw the dart by randomly generating a x and y coordinate
7      */
8      x = (double) drand48();
9      y = (double) drand48();
10
11     /*
12     *   If (x,y) is less than 1 away from the origin, the dart hit the quarter circle
13     */
14     if ( (x*x + y*y) <=1) // Technically we need to use a sqrt here, but sqrt(x) < 1 if and only if
x < 1 here
15     {
16         hits++; // count the hit
17     }
18
19 }
20
21 /*
22 * Number of hits / number of attempts approximates the area of the quarter circle.
23 * Multiply by 4 to get area of full circle.
24 */
25 pi = 4.0*( (double) hits / (double) attempts);
26
```

What can we define as **task** here?

What can we define as **task** here?

- Throw one dart: very, very quick to compute thus very small granularity. Overheads will likely be significant.
- Better: Throw N many darts. This allows to adjust N to some meaningful number and to control, in some sense, granularity.
- Need to do three steps:
 - Distribute the tasks: each thread/process needs to know its N
 - Perform N attempts
 - Sum up the number of hits and number of attempts from all threads/processes and compute approximate pi.

Every thread initializes the random number generator

```
1  #pragma omp parallel
2  {
3      srand48(omp_get_thread_num());
4      #pragma omp for private(x, y) reduction(+ : hits)
5      for (i=0; i < attempts; i++)
6      {
7          x = (double) drand48();
8          y = (double) drand48();
9          if ( (x*x + y*y) <=1)
10         {
11             hits++;
12         }
13     }
14 }
15
```

This tells OpenMP to sum (+) all the variables `hits` when the loop has finished. Because it reduces many numbers into one, it is called a reduction operation.

Each thread holds its own copies of the `x` and `y` coordinates

Every thread initializes the random number generator

```
1  #pragma omp parallel
2  {
3      srand48(omp_get_thread_num());
4      #pragma omp for private(x, y) reduction(+ : hits)
5      for (i=0; i < attempts; i++)
6      {
7          x = (double) drand48();
8          y = (double) drand48();
9          if ( (x*x + y*y) <=1)
10         {
11             hits++;
12         }
13     }
14 }
15
```

This tells OpenMP to sum (+) all the variables `hits` when the loop has finished. Because it reduces many numbers into one, it is called a reduction operation.

Each thread holds its own copies of the `x` and `y` coordinates

Try the code and see how it scales

Thread safety

A function is called **thread safe** if it can be called simultaneously by multiple threads without unintended side effects

Thread safety

A function is called **thread safe** if it can be called simultaneously by multiple threads without unintended side effects

- As it turns out, `drand48()` is not thread safe: it stores some internal variable in the global state for the random number generator that changes on each call

Thread safety

A function is called **thread safe** if it can be called simultaneously by multiple threads without unintended side effects

- As it turns out, `drand48()` is not thread safe: it stores some internal variable in the global state for the random number generator that changes on each call

```
1  X(n+1) = (aX(n) + c) mod (2**48)      n >= 0
2  //Initial values:
3  X(0) = 1
4  a    = 0x5DEECE66D (base 16)
5  c    = 0xB          (base 16)
6
```

Thread safety

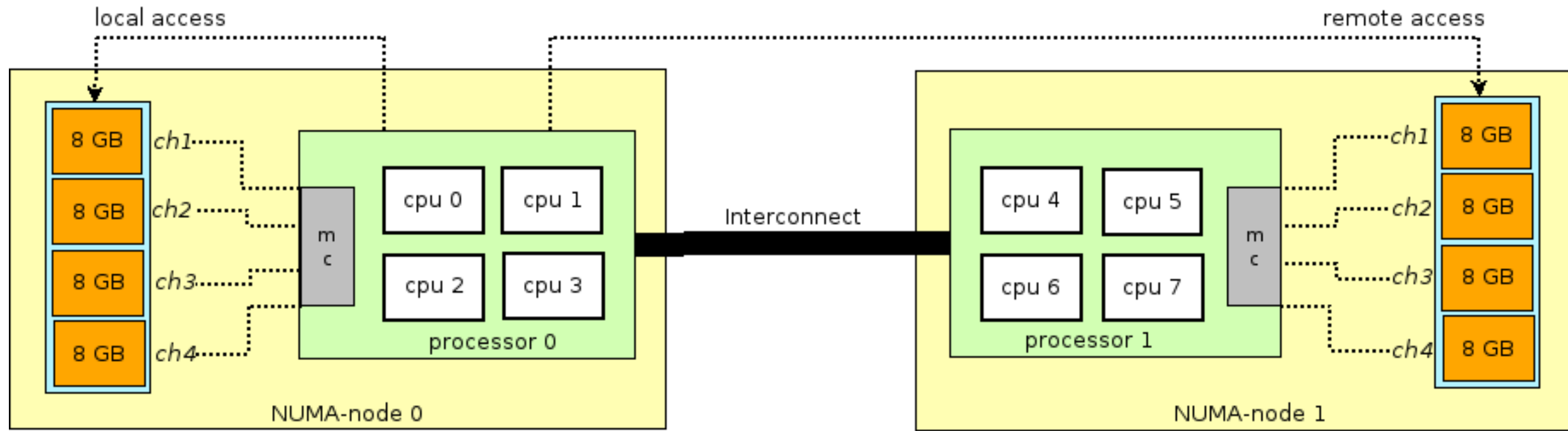
A function is called **thread safe** if it can be called simultaneously by multiple threads without unintended side effects

- As it turns out, `drand48()` is not thread safe: it stores some internal variable in the global state for the random number generator that changes on each call

```
1  X(n+1) = (aX(n) + c) mod (2**48)      n >= 0
2  //Initial values:
3  X(0) = 1
4  a    = 0x5DEECE66D (base 16)
5  c    = 0xB          (base 16)
6
```

- Hence, we need to use `erand48()` instead which uses a thread private state.
- Check out `mc_pi_omp.c` and `mc_pi_omp_threadsafe.c` and see how they scale

Non-uniform memory access (NUMA)



- NUMA : although logically there is one shared memory, speed of access depends on where in (physical) memory data is located
- cc-NUMA: every "client" (processor / core) holds a copy of a small segment of the main memory. The CC protocol makes sure all these copies are coherent" all the time

First Touch Placement Policy allocates the data in the memory closest to the thread accessing it for the first time
...make sure the thread that uses parts of an array is also the thread that initializes it.

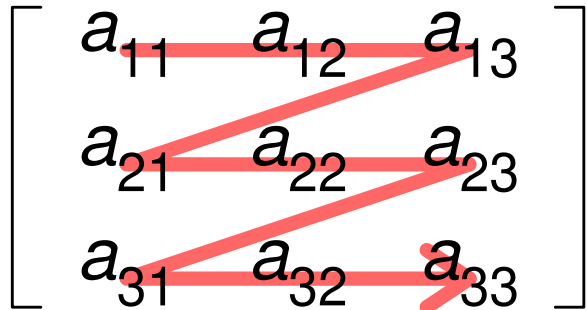
Initialization:

```
1  #pragma omp parallel for private(i, j)
2  for (i=0; i<rows; i++)
3  {
4      for (j=0; j<columns; j++)
5      {
6          a[i][j] = 1.0;
7      }
8      x[i] = 1.0;
9  }
10
```

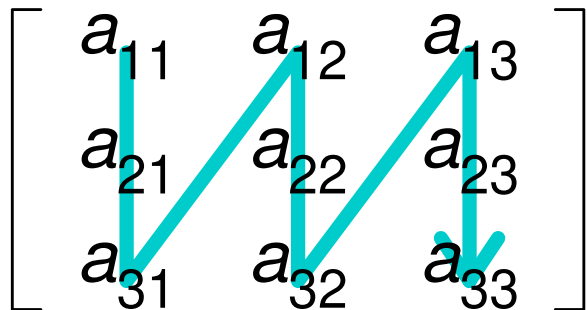
Computation:

```
1  #pragma omp parallel for private(i, j, sum)
2  for (i=0; i<rows; i++)
3  {
4      sum = 0.0;
5      for (j=0; j<columns; j++)
6      {
7          sum += a[i][j]*x[j];
8      }
9      b[i] = sum;
10 }
11
```

Row-major order



Column-major order



- C uses row-major order
- FORTRAN and MATLAB use column-major
- In row-major order, nested loops should have the column counter as inner loop to make sure data is accessed in the order it is stored in memory

```
1 for (i=0; i < rows; i++)  
2   for (j=0; j < columns; j++)  
3     a[i][j] = ...  
4
```

← use this in C

```
1 for (j=0; j < columns; j++)  
2   for (i=0; i < rows; i++)  
3     a[i][j] = ...  
4
```

← use this in Fortran

You should now be familiar with the following terms:

- Monte Carlo Approximation of Pi in OpenMP
- Thread safety
- NUMA
- First touch policy

Homework

Run the “large_example” for the matrix vector OpenMP code and analyse performance!